

EduGenie

Google Gemini Powered Learning Assistant

Complete Step-by-Step Project Work Guide

This guide is designed for a college-level AI/Generative AI mini project. It covers the complete development process from environment setup to testing and presentation.

Project	EduGenie - Google Gemini Powered Learning Assistant
Main Technology	Python + Web UI + Google Gemini API
AI Role	Question answering, explanation, summarization, quiz generation and study assistance
Suggested Backend	Flask
Frontend	HTML + CSS + JavaScript
Configuration	.env file for Gemini API key

1. Project Objective

EduGenie is an AI-based learning assistant that helps students understand academic topics using natural-language interaction. A student can enter a question or topic and receive an explanation generated by Google Gemini.

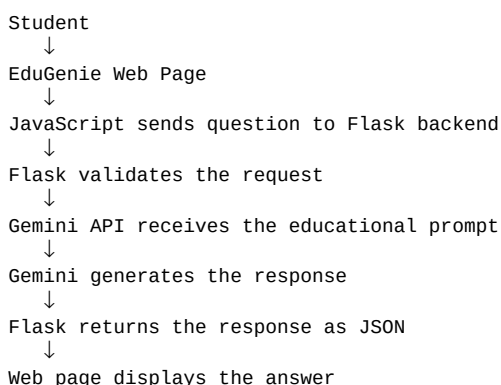
The application can be extended with features such as simple explanations, summaries, examples, quizzes, study plans, and revision questions.

2. Main Features

- Ask EduGenie a question in normal language.
- Generate beginner-friendly explanations.
- Summarize a topic or supplied notes.
- Generate examples and key points.
- Create practice questions or MCQs.
- Generate a simple study plan for a selected topic.
- Display the AI response inside a clean web interface.
- Keep the Gemini API key outside the source code using environment variables.

3. System Workflow

The basic workflow is:



4. Software Requirements

- Windows 10/11 or another supported desktop operating system.
- Python 3.x installed.
- Visual Studio Code.
- A Google account for obtaining Gemini API access.
- Internet connection while using the Gemini API.
- A modern web browser such as Chrome or Edge.

5. Create the Project Folder

Create a folder named EduGenie. Open that folder in VS Code.

```
EduGenie/  
■■■ app.py
```

```

■■■ requirements.txt
■■■ .env
■■■ .gitignore
■■■ templates/
■   ■■■ index.html
■■■ static/
    ■■■ css/
    ■   ■■■ style.css
    ■■■ js/
        ■■■ script.js

```

6. Create a Virtual Environment

Open the VS Code terminal inside the EduGenie folder and run:

```
python -m venv venv
```

Activate it in PowerShell:

```
venv\Scripts\Activate.ps1
```

If PowerShell blocks script execution, use Command Prompt and run:

```
venv\Scripts\activate.bat
```

7. Install Required Packages

Create requirements.txt:

```

Flask
python-dotenv
google-genai

```

Then install the packages:

```
pip install -r requirements.txt
```

8. Gemini API Setup

Create or access your Google AI/Gemini developer environment, create an API key according to the current Google instructions, and keep the key private.

In the project root, create a file named .env:

```
GEMINI_API_KEY=PASTE_YOUR_API_KEY_HERE
```

Do not put quotation marks around the key unless the platform specifically requires them.

9. Protect the API Key

Create .gitignore so the secret is not accidentally committed to a Git repository:

```

.env
venv/
__pycache__/
*.pyc

```

Never upload or share the real API key in screenshots, reports, public repositories, or chat messages.

10. Backend - app.py

Create app.py and use this starter implementation. The model name can be changed to a currently available Gemini model if required by your API account.

```

import os
from flask import Flask, render_template, request, jsonify
from dotenv import load_dotenv
from google import genai

load_dotenv()

```

```

app = Flask(__name__)

api_key = os.getenv("GEMINI_API_KEY")
if not api_key:
    raise RuntimeError("GEMINI_API_KEY is missing in .env")

client = genai.Client(api_key=api_key)

MODEL_NAME = "gemini-2.5-flash"

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/ask", methods=["POST"])
def ask():
    data = request.get_json(silent=True) or {}
    question = (data.get("question") or "").strip()

    if not question:
        return jsonify({"error": "Please enter a question."}), 400

    prompt = f"""
You are EduGenie, a friendly educational learning assistant.
Explain the following question clearly for a college student.
Use simple language, short sections, examples where useful,
and avoid unnecessary complexity.

Question:
{question}
"""

    try:
        response = client.models.generate_content(
            model=MODEL_NAME,
            contents=prompt
        )
        return jsonify({"answer": response.text})
    except Exception as exc:
        return jsonify({"error": str(exc)}), 500

if __name__ == "__main__":
    app.run(debug=True)

```

If the selected model is unavailable for your account, replace MODEL_NAME with a currently supported Gemini model available in your Google AI environment.

11. Frontend - templates/index.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>EduGenie</title>
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
  <main class="container">
    <h1>EduGenie</h1>
    <p class="tagline">Your Google Gemini Powered Learning Assistant</p>

    <textarea id="question"
      placeholder="Ask a question or enter a topic..."></textarea>

    <button id="askBtn">Ask EduGenie</button>

    <section id="result" class="result">
      <h2>Answer</h2>
      <div id="answer">Your answer will appear here.</div>
    </section>
  </main>

  <script src="/static/js/script.js"></script>
</body>
</html>

```

12. Frontend Styling - static/css/style.css

```
body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f4f6fb;
  color: #222;
}

.container {
  width: min(850px, 90%);
  margin: 50px auto;
  background: white;
  padding: 30px;
  border-radius: 16px;
  box-shadow: 0 8px 25px rgba(0,0,0,0.08);
}

h1 {
  margin-bottom: 5px;
}

.tagline {
  color: #666;
}

textarea {
  width: 100%;
  min-height: 150px;
  padding: 12px;
  margin: 20px 0;
  box-sizing: border-box;
  border: 1px solid #ccc;
  border-radius: 10px;
  resize: vertical;
}

button {
  padding: 12px 20px;
  border: 0;
  border-radius: 10px;
  cursor: pointer;
  font-weight: bold;
}

.result {
  margin-top: 25px;
  padding: 18px;
  border-radius: 10px;
  background: #f7f7f7;
  white-space: pre-wrap;
}
```

13. JavaScript - static/js/script.js

```
const questionBox = document.getElementById("question");
const askBtn = document.getElementById("askBtn");
const answerBox = document.getElementById("answer");

askBtn.addEventListener("click", async () => {
  const question = questionBox.value.trim();

  if (!question) {
    answerBox.textContent = "Please enter a question.";
    return;
  }

  askBtn.disabled = true;
  askBtn.textContent = "Thinking...";
  answerBox.textContent = "EduGenie is generating an answer...";

  try {
    const response = await fetch("/ask", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      }
    });
  }
});
```

```

        },
        body: JSON.stringify({ question })
    });

    const data = await response.json();

    if (!response.ok) {
        throw new Error(data.error || "Request failed.");
    }

    answerBox.textContent = data.answer;
} catch (error) {
    answerBox.textContent = "Error: " + error.message;
} finally {
    askBtn.disabled = false;
    askBtn.textContent = "Ask EduGenie";
}
});

```

14. Run the Project

Make sure the virtual environment is active. In the VS Code terminal run:

```
python app.py
```

You should see Flask start the local server. Open the local address shown in the terminal, normally something similar to `http://127.0.0.1:5000`.

Do not type the URL into the Python terminal. Open it in Chrome or another browser.

15. First Test

Enter a simple question such as: What is cloud computing? Explain it with a real-life example.

Click Ask EduGenie. The browser sends the question to Flask, Flask sends the prompt to Gemini, and the generated answer is displayed on the page.

16. Common Errors and Fixes

Error	What to check
ModuleNotFoundError	Activate venv and run <code>pip install -r requirements.txt</code>
GEMINI_API_KEY is missing	Check that <code>.env</code> is in the same project root as <code>app.py</code> and the variable name is exact.
401/403 API error	Check the API key, project/API access, restrictions, and current Google configuration.
404 page	Check that the Flask app is running and the browser uses the correct local address.
500 Internal Server Error	Read the traceback in the VS Code terminal; it usually identifies the failing line.
No answer in browser	Open browser developer tools and check the Network/Console messages; also inspect the Flask terminal

17. Recommended Project Enhancements

- Add separate buttons for Explain, Summarize, Quiz and Study Plan.
- Add a difficulty selector: Beginner, Intermediate, Advanced.
- Add subject selection such as Python, Java, DBMS, Computer Networks, Mathematics, etc.
- Add chat history using a database such as SQLite.
- Add loading animation and better error messages.
- Add text-to-speech for generated explanations.

- Add a quiz score screen.
- Add an admin/dashboard page if required by the college project.

18. Suggested Advanced Prompt Design

Instead of sending only the student's raw question, the backend can classify the requested learning action and create a structured prompt. Example:

```
Role: You are EduGenie, an educational assistant.
Student level: College beginner.
Task: Explain the topic.
Topic: {student_question}
```

```
Response format:
1. Simple definition
2. Key points
3. Real-life example
4. Short example
5. 3 revision questions
```

Keep the explanation accurate and easy to understand.

19. Project Modules

Module	Purpose
User Interface	Collects the student's question and displays the answer.
Flask Backend	Handles browser requests and application logic.
Prompt Layer	Converts the student's request into an educational instruction.
Gemini Integration	Sends the prompt to Gemini and receives generated content.
Response Layer	Returns the generated answer to the browser.
Security Layer	Keeps API credentials in environment variables.

20. Testing Checklist

- ☐ Python installation verified.
- ☐ VS Code project folder created.
- ☐ Virtual environment created and activated.
- ☐ Dependencies installed successfully.
- ☐ Gemini API key configured in .env.
- ☐ .env added to .gitignore.
- ☐ Flask server starts without errors.
- ☐ Home page opens in the browser.
- ☐ Empty question validation works.
- ☐ Normal educational question returns an answer.
- ☐ API/server error is displayed clearly.
- ☐ Final UI works on both desktop and mobile-sized browser windows.

21. Demonstration Questions

- Explain object-oriented programming in simple words.
- What is a database? Give a real-life example.
- Summarize the concept of cloud computing in five points.
- Create five MCQs about computer networks with answers.
- Explain polymorphism in C++ with a simple example.

22. Project Report Structure

For the college documentation, use this order:

1. Title Page
2. Certificate / Declaration (if required by college)
3. Acknowledgement
4. Abstract
5. Introduction
6. Problem Statement
7. Objectives
8. Existing System
9. Proposed System
10. System Requirements
11. System Architecture
12. Module Description
13. Technologies Used
14. Implementation
15. Screenshots
16. Testing
17. Results
18. Advantages
19. Limitations
20. Future Scope
21. Conclusion
22. References

23. Short Abstract for the Report

EduGenie is a Google Gemini powered learning assistant designed to support students during self-learning. The system accepts natural-language questions through a web interface and uses a Gemini generative AI model to produce clear educational responses. The application can be extended to provide explanations, summaries, examples, quizzes and study plans. A Python Flask backend manages requests while HTML, CSS and JavaScript provide the user interface. The API credential is stored in an environment variable to avoid exposing it in the source code.

24. Advantages

- Interactive learning through natural-language questions.
- Fast generation of explanations and practice material.
- Can support multiple academic subjects.
- Simple web interface suitable for students.
- Can be expanded with additional AI-powered learning features.

25. Limitations

- Generated answers may occasionally contain mistakes and should be verified for important academic work.
- The application depends on API availability and internet access.
- API usage may be subject to account limits or billing rules.
- The quality of the output depends on the prompt and model used.

26. Future Scope

- Personalized learning paths based on student performance.
- Automatic quiz evaluation and progress tracking.
- PDF/notes upload and question answering from study material.
- Voice-based learning assistant.
- Multilingual explanations.
- Teacher/admin dashboard.
- Learning analytics and revision recommendations.

27. Final Project Flow for Viva

```
graph TD
    A[Student enters a question] --> B[Frontend collects the question]
    B --> C[JavaScript sends POST request]
    C --> D[Flask receives and validates it]
    D --> E[Prompt is created]
    E --> F[Gemini API generates educational content]
    F --> G[Flask sends JSON response]
    G --> H[Frontend displays the answer]
    H --> I[Student continues learning]
```

One-line viva explanation: “EduGenie is a web-based learning assistant where a Flask backend connects the student interface to Google Gemini, which generates personalized educational responses from the student's questions.”

28. Important Safety & Security Notes

Keep the Gemini API key private. Do not hard-code it into public JavaScript or upload the .env file to a public repository. For a college demonstration, use a restricted/test key where appropriate and monitor API usage. Also treat generated educational content as AI-generated material that may need verification.

29. Final Checklist Before Submission

- ☐ Project runs from a fresh terminal.
- ☐ requirements.txt is included.
- ☐ .env is excluded from public sharing.
- ☐ Screenshots of the working application are captured.
- ☐ Architecture/workflow diagram is included in the report.
- ☐ Testing table is completed.
- ☐ Future scope and limitations are documented.
- ☐ Demo questions are prepared for viva.